

TEAMMATE AUDIT APPLICATION

End-to-End Request Flow

How Every User Request Travels Through the System

AWS EKS | Kubernetes | Docker | Jenkins | Spring Boot
Redis | Kafka | MySQL | AWS S3 | Datadog | Splunk

Developer DevOps Reference Guide | 2025

Chapter 0 — Big Picture Overview

Every user request follows a predictable lifecycle. Before exploring each layer in detail, understand the complete flow in one view.

Stage	Who / What Acts	Tool / Technology
1. User opens browser	End user	Browser or Mobile App
2. DNS lookup	Route 53	AWS Route 53 → resolves to ALB IP
3. TLS handshake + route	AWS ALB + Ingress Controller	HTTPS/TLS, path-based routing
4. Request enters EKS cluster	Kubernetes Ingress	Ingress object → Service → Pod
5. Auth service validates token	Auth Service Pod	Spring Boot, JWT / OAuth2
6. Business logic executes	Audit / Rule Engine Pod	Spring Boot microservice
7. Cache checked	Redis	Session data, hot audit records
8. DB read / write	MySQL on AWS RDS	JPA / Hibernate via Spring Boot
9. Document stored / retrieved	AWS S3	AWS SDK from Spring Boot
10. Async event published	Kafka	Audit → Notification service
11. Notification sent	Notification Service Pod	Email / SMS / push
12. Response returns	Reverse path through K8s	JSON over HTTPS
13. Logs shipped	Fluent Bit sidecar	Splunk centralized logging
14. Metrics emitted	Datadog Agent sidecar	Datadog APM + dashboards

Interview tip: Always say — 'Our pipeline builds a Docker image once in the dev branch and promotes the same artifact through DEV → QA → UAT → PROD via Kubernetes namespaces. Environment differences are injected via ConfigMaps and Secrets, never baked into the image.'

Chapter 1 — User Layer to Ingress

The journey begins the moment a user opens their browser or mobile app. This chapter explains every hop before the request reaches a Kubernetes pod.

1.1 DNS Resolution — AWS Route 53

When the user types `https://teammate.com`, the browser performs a DNS lookup.

- Route 53 is AWS's managed DNS service.
- It resolves the domain to the IP address of the Application Load Balancer (ALB).
- For production resilience, Route 53 can be configured with health checks and failover routing policies.
- Latency-based routing can direct global users to the nearest AWS region.

1.2 TLS Termination — AWS ALB

The request arrives at the Application Load Balancer over HTTPS (port 443).

- ALB terminates the TLS connection. The SSL certificate lives here, managed by AWS Certificate Manager (ACM).
- From this point inward, traffic travels inside the AWS VPC — typically over HTTP or internal HTTPS.
- ALB evaluates path-based routing rules: `/api/reporting/*` → reporting service, `/api/auth/*` → auth service, etc.
- ALB forwards the request to the Kubernetes Ingress Controller (usually running as a pod inside EKS).

1.3 Kubernetes Ingress Controller

The Ingress Controller (e.g., NGINX or AWS ALB Ingress Controller) reads Ingress objects defined in YAML.

Ingress Rule	Routed To
<code>/api/auth/**</code>	auth-service (ClusterIP) → Auth Pods
<code>/api/audit/**</code>	audit-service (ClusterIP) → Audit Pods
<code>/api/reporting/**</code>	reporting-service (ClusterIP) → Reporting Pods
<code>/api/notifications/**</code>	notification-service (ClusterIP) → Notification Pods
<code>/api/rules/**</code>	rule-engine-service (ClusterIP) → Rule Engine Pods

The Kubernetes Service (type: ClusterIP) is an internal stable IP + DNS name. It load-balances across all healthy pods for that service. Pods come and go — the Service IP stays constant.

Chapter 2 — Inside the EKS Cluster

Once the request crosses the Ingress boundary, it is inside the AWS EKS (Elastic Kubernetes Service) cluster. EKS manages the Kubernetes control plane — you only manage worker nodes.

2.1 EKS Cluster Architecture

The cluster consists of a control plane (managed by AWS) and EC2 worker nodes (managed by you).

K8s Component	Location	Role in Teammate
API Server	Control Plane (AWS managed)	Accepts all kubectl and Jenkins commands
etcd	Control Plane (AWS managed)	Stores desired state (deployments, replicas, config)
Scheduler	Control Plane (AWS managed)	Assigns new pods to the best available worker node
Controller Manager	Control Plane (AWS managed)	Ensures actual state matches desired state; restarts failed pods
kubelet	Each EC2 Worker Node	Runs and monitors containers on that node
kube-proxy	Each EC2 Worker Node	Handles pod-to-pod network routing
Docker Engine	Each EC2 Worker Node	Actually executes the container images

2.2 Worker Nodes and Pod Placement

Worker nodes are EC2 instances (t2.medium). Teammate runs two nodes minimum, scaled by Cluster Autoscaler when CPU or memory crosses threshold.

- Node 1: Auth Service, Audit Service, Reporting Service
- Node 2: Notification Service, Rule Engine Service, Background Jobs Service
- The Kubernetes Scheduler places pods based on resource requests and node affinity rules.
- If a node fails, the Scheduler reschedules its pods on remaining healthy nodes — automatic.

2.3 Pod Structure — App Container + Sidecar

Every microservice pod in Teammate contains two containers sharing the same network namespace and volumes.

Container	Purpose
Spring Boot App Container	Runs the actual Java application on port 8080. Reads config from env vars injected by Kubernetes ConfigMaps and Secrets.

Container	Purpose
Sidecar: Datadog Agent	Collects application metrics, distributed traces (APM), and forwards them to Datadog cloud.
Sidecar: Fluent Bit	Tails the shared log volume or stdout/stderr, parses log lines, and ships them to Splunk.
Shared Volume	Mounted by both containers. App writes logs to /var/log/app.log; Fluent Bit reads from same path.

2.4 ConfigMap and Secrets — Environment Injection

The same Docker image runs in all environments (DEV, QA, UAT, PROD). Environment-specific values are injected at pod startup time.

Object	Stores
ConfigMap: teammate-config	DB URL, Redis host, S3 bucket name, Kafka bootstrap servers, log level
Secret: teammate-secrets	DB password, API tokens, AWS credentials, JWT signing key

Each Kubernetes namespace (teammate-dev, teammate-qa, teammate-prod) has its own ConfigMap and Secret with environment-appropriate values. The Spring Boot application.properties uses placeholder variables like \${DB_URL} which Kubernetes replaces at runtime.

Chapter 3 — Request Execution Inside Spring Boot

Once Kubernetes routes the request to the correct pod, Spring Boot takes over. This chapter traces execution through the layers of the application.

3.1 Spring Boot Request Lifecycle

Step
1

Embedded Tomcat receives request

Spring Boot's embedded Tomcat listens on port 8080 inside the container. It accepts the HTTP request forwarded by the Kubernetes Service.

Step
2

Spring Security filter chain

Every request passes through the security filter chain. JWT tokens are validated against the signing key (injected from Kubernetes Secret). Invalid tokens return 401 immediately.

Step
3

@RestController handles request

The request is dispatched to the matching @RequestMapping method. Input is deserialized from JSON via Jackson. @Valid annotations trigger bean validation.

Step
4

Service layer + business logic

@Service classes execute business rules. For audit actions, the Rule Engine is called synchronously via internal Kubernetes DNS (rule-engine-service.teammate-prod.svc.cluster.local).

Step
5

Redis cache lookup

Before hitting the database, the service checks Redis for cached data (hot audit records, user sessions). Cache hit = return immediately. Cache miss = proceed to DB.

Step
6

MySQL database query

Spring Data JPA / Hibernate executes the SQL via connection pool (HikariCP). The DB URL comes from the ConfigMap; password from the Secret. AWS RDS handles replication and failover.

Step
7

S3 document access (if applicable)

Audit document uploads/downloads use the AWS SDK. The S3 bucket name is environment-specific (teammate-audit-prod). The IAM role on the EC2 node grants S3 access without hardcoded credentials.

**Step
8****Kafka event published (async)**

On completing an audit action, the service publishes an event to Kafka (e.g., `audit.completed`). This is fire-and-forget — the HTTP response is not held. The Notification Service consumes this event independently.

**Step
9****Response serialized and returned**

The `@RestController` returns the result object. Jackson serializes it to JSON. The HTTP 200 response travels back through the Kubernetes Service, Ingress, ALB, and TLS to the user's browser.

Zero-downtime deployments: When Jenkins deploys build #47, Kubernetes starts new pods, waits for `/actuator/health` to return healthy (readiness probe), then terminates old pods. Users never hit an unhealthy pod. `kubectl rollout undo` instantly reverts if needed.

Chapter 4 — Supporting Infrastructure

Behind the microservices sits a set of shared infrastructure components. Each solves a distinct problem in the system.

4.1 Redis — Caching Layer

- What it caches: User sessions, frequently accessed audit record lookups, permission matrices.
- Why: Database queries for hot data are expensive. Redis responds in microseconds.
- Spring Boot integration: Spring Cache abstraction with `@Cacheable`, `@CacheEvict` annotations.
- TTL (time-to-live): Session cache = 30 min, audit record cache = 5 min, configurable via ConfigMap.
- In Kubernetes: Redis runs as a ClusterIP service accessible only within the cluster. Auth service and Audit service both connect to the same Redis instance.

4.2 Kafka — Async Event Bus

Kafka decouples services that do not need to wait for each other.

Kafka Topic	Producer → Consumer
audit.completed	Audit Service → Notification Service (email to auditor)
audit.flagged	Rule Engine → Audit Service (flag for review)
document.uploaded	Audit Service → Background Jobs (virus scan, OCR)
report.requested	Audit Service → Reporting Service (async PDF generation)

- Producers publish events and immediately return to the caller. No blocking.
- Consumers process events at their own rate. Notification Service sends emails without slowing the Audit Service.
- Kafka retains events for 7 days. If Notification Service is down, it processes the backlog on restart.

4.3 MySQL on AWS RDS — Primary Database

- Managed by AWS RDS — automatic backups, Multi-AZ failover, automated patching.
- Connection string injected via ConfigMap per namespace (dev DB, QA DB, prod DB are separate RDS instances).
- HikariCP connection pool in Spring Boot — configured with max pool size, connection timeout.
- Each environment's RDS Security Group only allows inbound connections from the EKS worker node security group.

4.4 AWS S3 — Document Storage

- Auditors upload PDF documents and evidence files. These never go into MySQL (binary in DB is an anti-pattern).
- Spring Boot uses the AWS Java SDK v2 to call S3 PutObject (upload) and GetObject (download).
- Bucket names are environment-specific: teammate-audit-dev, teammate-audit-prod.
- Pre-signed URLs are generated for secure temporary access without exposing bucket credentials to the browser.
- The IAM role attached to EKS worker nodes grants s3:PutObject and s3:GetObject on the appropriate bucket.

Chapter 5 — Monitoring & Logging

Observability is built into every pod via sidecars. You should know this flow for debugging in production.

5.1 Logging Flow — Fluent Bit → Splunk

- Spring Boot writes structured logs (JSON format) to stdout/stderr and to a shared volume.
- Fluent Bit sidecar container reads from that shared volume (or Docker's log driver).
- Fluent Bit parses, enriches (adds pod name, namespace, service label), and forwards to Splunk.
- In Splunk: Search index=teammate-prod service=audit-service level=ERROR to find issues.
- Fluent Bit buffers logs locally if Splunk is unavailable — no log loss during Splunk restarts.

5.2 Metrics & APM — Datadog Agent

- Datadog Agent sidecar scrapes JVM metrics (heap, GC, thread count) every 15 seconds.
- Spring Boot Actuator exposes /actuator/metrics — Datadog reads these via the JMX integration.
- Distributed tracing: Datadog APM agent instruments Spring Boot automatically. Every HTTP call between microservices gets a trace ID. You can visualize the full request path across Auth → Audit → Rule Engine → DB.
- Custom dashboards: Request rate, error rate, p99 latency per service — all visible in Datadog.

5.3 Health Checks — Kubernetes Probes

Kubernetes continuously polls two endpoints on every pod to know if it is healthy.

Probe Type	Behaviour
Readiness Probe — GET /actuator/health	If this returns non-200, the pod is removed from the Service load balancer. Traffic stops going to it. Used during rolling deployments to ensure old traffic isn't routed to an initializing pod.
Liveness Probe — GET /actuator/health	If this returns non-200 for N consecutive checks, Kubernetes restarts the container. Catches infinite loops and deadlocks.

Spring Boot Actuator is the key enabler. Add spring-boot-starter-actuator dependency and expose the health endpoint via management.endpoints.web.exposure.include=health in application.properties.

5.4 Prometheus + Grafana (Optional)

- Prometheus scrapes /actuator/prometheus endpoint (exposed via micrometer-registry-prometheus dependency) every 30 seconds.
- Grafana connects to Prometheus as a data source and renders dashboards.

- Alerts fire to PagerDuty (or Slack) when thresholds are breached — e.g., error rate > 5% for 5 minutes.

Chapter 6 — CI/CD Pipeline: Code to Production

This chapter explains how your code on a laptop becomes a running container in the AWS EKS production cluster.

6.1 Pipeline Steps in Order

Step
1

Developer writes code

Feature developed in IDE. Tested locally with `java -jar` or from IDE. Spring Boot runs on `localhost:8080`. No Docker needed locally.

Step
2

git push to feature branch

`git checkout dev && git pull` to get latest. Then `git checkout -b feature/audit-export` and `git push origin feature/audit-export`.

Step
3

Pull Request: feature → dev

PR created via GitHub UI. Code review by teammates. On approval and merge, GitHub sends a webhook to Jenkins on VM-2.

Step
4

Jenkins pipeline triggers

Jenkins (running on its own EC2 VM — VM-2) receives the webhook. It reads the Jenkinsfile in the repository root. The Jenkinsfile defines all pipeline stages as code.

Step
5

Maven Build + Tests

Jenkins runs: `mvn clean package`. This compiles Java, runs all `@Test` unit tests, and produces the JAR in `target/`. Failure here stops the pipeline — broken code never gets packaged.

Step
6

SonarQube code quality scan

Jenkins runs: `mvn sonar:sonar`. SonarQube checks code coverage, code smells, security vulnerabilities, and duplications. Results visible in the SonarQube dashboard. Can be configured to fail the pipeline if coverage drops below threshold.

Step
7

Docker image build

Jenkins reads the Dockerfile in the repo root. Runs: `docker build -t ecr-url/reporting-service:BUILD_47`. The image uses `openjdk:17-jdk-slim` as base, copies the JAR, exposes port 8080.

**Step
8****Push image to AWS ECR**

Jenkins authenticates to ECR: `aws ecr get-login-password | docker login`. Then: `docker push ecr-url/reporting-service:BUILD_47`. ECR is like Docker Hub but private and inside your AWS account.

**Step
9****Deploy to EKS namespace**

Jenkins runs: `kubectl set image deployment/reporting-reporting-service=ecr/reporting:BUILD_47 -n teammate-dev`. Kubernetes begins the rolling update — new pods with BUILD_47 start, old pods stop after health checks pass.

**Step
10****Promote through environments**

Same image (BUILD_47) is promoted: dev namespace → QA namespace (after QA sign-off) → UAT namespace (manual approval gate) → PROD namespace (second approval). No rebuild. Guaranteed identical binary in all envs.

6.2 Branching Strategy

Git Branch	Kubernetes Namespace	Jenkins Behaviour
dev	teammate-dev	Auto-trigger on every push. Deploy immediately. Fast feedback loop for developers.
main	teammate-qa	Trigger on PR merge from dev. Full test suite runs. QA team tests here.
release	teammate-uat	Manual approval gate in Jenkins. Same image from QA promoted. Client UAT testing.
prod tag	teammate-prod	Second manual approval. Same image promoted. Zero-downtime rolling update.

Chapter 7 — Full Scenario: New API End to End

Trace the complete journey of the Audit Summary Export API feature — from the developer's keyboard to an auditor's browser.

7.1 Development Phase

- Developer adds GET `/api/reporting/audit-summary/{auditId}` endpoint to ReportingController.
- New `@Service` method queries MySQL for audit data, generates a summary PDF, uploads it to S3, and returns the S3 pre-signed URL.
- Unit tests added for the service layer. Integration test added for the controller.
- Tested locally with `java -jar reporting-service.jar` and Postman.

7.2 CI/CD Phase

- `git push origin feature/audit-summary-export`.
- PR created: feature → dev. Code review done. Merged.
- Jenkins pipeline runs automatically. Maven builds. Tests pass. SonarQube passes. Docker image built as `reporting-service:BUILD_47`.
- Image pushed to ECR. Jenkins deploys to `teammate-dev` namespace via `kubectl`.
- Rolling update completes. QA tests the API at `dev.teammate.com/api/reporting/audit-summary/123`.

7.3 Production Release Phase

- PR merged: dev → main. Jenkins deploys `BUILD_47` to `teammate-qa`. QA team validates.
- PR merged: main → release. Jenkins pauses for manual approval. DevOps approves. `BUILD_47` deployed to `teammate-uat`. Client tests.
- Approval granted for PROD. Jenkins runs `kubectl apply` with `BUILD_47` in `teammate-prod` namespace.
- Kubernetes rolling update: 3 new reporting pods with `BUILD_47` start, readiness probe passes, 3 old pods terminate.
- Auditors open `https://teammate.com` and use the new export API. The same JAR that passed QA is what runs in PROD.

7.4 What Happens at Runtime (Request Flow)

Auditor clicks Export Summary in the browser:

Hop	What Happens
Browser → Route 53	DNS resolves <code>teammate.com</code> to ALB IP
Route 53 → ALB	HTTPS request arrives, TLS terminated
ALB → Ingress	Path <code>/api/reporting</code> matched, forwarded to Ingress Controller

Hop	What Happens
Ingress → reporting-service (ClusterIP)	Service load-balances to one of 3 Reporting pods
Reporting Pod — Auth filter	JWT validated via Spring Security (key from Secret)
Reporting Pod — Controller	GET /audit-summary/123 dispatched to ReportingController
Reporting Pod — Cache check	Redis queried: key audit:summary:123. Cache miss.
Reporting Pod — DB query	JPA queries MySQL RDS for audit record 123
Reporting Pod — S3 upload	PDF generated, uploaded to teammate-audit-prod S3 bucket
Reporting Pod — Kafka publish	report.generated event published (async, non-blocking)
Reporting Pod → Response	Pre-signed S3 URL returned as JSON with 200 OK
Response → Browser	Reverse path. Auditor's browser downloads PDF from S3 URL.
Fluent Bit → Splunk	Request log line shipped to Splunk within seconds
Datadog Agent → Datadog	APM trace recorded with full service graph and latency breakdown

Chapter 8 — Troubleshooting Quick Reference

When something breaks, here is your step-by-step playbook. Know these for your daily work and for interviews.

8.1 Jenkins Pipeline Failures

Failing Stage	First Thing to Check
Build & Test (mvn package)	Jenkins console output for compilation errors. Missing dependency in pom.xml? Test failures in a specific class?
SonarQube	Code coverage below threshold? New critical vulnerabilities? Check SonarQube dashboard directly.
Docker Build	Does target/*.jar exist? Did mvn package succeed? COPY path in Dockerfile matches exact JAR name?
Push to ECR	ECR auth token expired (expires every 12 hours). Ensure aws ecr get-login-password step runs before docker push.
Deploy to K8s	kubectl Unauthorized? kubeconfig credential in Jenkins may be stale. DevOps refreshes it.

8.2 Pod Issues in Kubernetes

`kubectl get pods -n teammate-prod` — check STATUS column first.

Pod Status	Diagnosis Approach
CrashLoopBackOff	<code>kubectl describe pod <name> -n teammate-prod</code> to see events. <code>kubectl logs <name> --previous</code> to see last crash reason. Check for missing env vars (DB_URL), wrong port, OOM kill.
Pending	<code>kubectl describe pod</code> : Is there a node with enough CPU/memory? Is the ECR image pullable (auth issue)?
OOMKilled	Container exceeded resources.limits.memory. Increase in deployment.yaml or optimize app memory.
ImagePullBackOff	ECR image tag doesn't exist, or worker node lacks ECR pull permission (IAM role issue).
0/3 Running	All replicas failing — urgent. <code>kubectl logs -l app=reporting-service -n teammate-prod</code> to check all pods at once.

8.3 Production Rollback

If the new deployment is broken:

- Instant rollback: `kubectl rollout undo deployment/reporting-service -n teammate-prod`

- This switches K8s to run the previous image — no rebuild, no pipeline. Under 30 seconds.
- Check history: `kubectl rollout history deployment/reporting-service -n teammate-prod`
- Rollback to specific revision: `kubectl rollout undo deployment/reporting-service --to-revision=45 -n teammate-prod`

Golden rule for production debugging: 1) Check Splunk for ERROR logs first. 2) Check Datadog APM for which service has the latency spike or error rate spike. 3) `kubectl describe pod` for Kubernetes-level issues. 4) `kubectl rollout undo` as the emergency lever.

Chapter 9 — Interview Cheat Sheet

Use these answers verbatim or adapt them. Each is tailored to the Teammate project.

Docker

Q: What is Docker and why do you use it?

We containerize each microservice to ensure the JAR that passes QA is byte-for-byte identical to what runs in production. Each service has a Dockerfile using `openjdk:17-jdk-slim` as the base image. The Maven-built JAR is copied in and started via `ENTRYPOINT [java, -jar, app.jar]`. We never bake environment-specific config into the image — Kubernetes injects it at runtime via `ConfigMaps` and `Secrets`.

Jenkins / CI-CD

Q: Explain your CI/CD pipeline.

Our Jenkinsfile-driven pipeline has these stages: Git checkout → Maven build + unit tests → SonarQube scan → Docker build → ECR push → `kubectl` rollout. Jenkins lives on its own EC2 VM (VM-2), separate from the app server (VM-1), to avoid resource contention. We build the Docker image once in the dev branch and promote that exact artifact through all environments.

Kubernetes

Q: How does zero-downtime deployment work?

Kubernetes uses a rolling update strategy by default. When Jenkins deploys build #47, K8s starts new pods, waits for the readiness probe at `/actuator/health` to return 200, and only then terminates old pods. If new pods fail health checks, the rollout stops automatically — old pods keep serving traffic. We can also `kubectl rollout undo` to instantly revert.

Q: How do services communicate inside Kubernetes?

Synchronous calls use Spring Boot `RestTemplate` or `WebClient` with Kubernetes internal DNS names: `reporting-service.teammate-prod.svc.cluster.local`. Services that do not need an answer immediately use Kafka — the Audit Service publishes an event and returns immediately; the Notification Service consumes it asynchronously.

Architecture

Q: Why did you extract these specific microservices?

We extracted Reporting (resource-intensive, heavy PDF generation), Document Management (needs isolated S3 access), Notifications (event-driven, better as a Kafka consumer), and Background Jobs (needs independent scheduling without blocking the monolith thread pool). Core audit workflow logic stayed in the monolith to preserve transactional consistency during the migration phase.

9.1 Quick Command Reference

Scenario	Command
See all pods in prod	<code>kubectl get pods -n teammate-prod</code>
Stream live logs	<code>kubectl logs <pod-name> -n teammate-prod -f</code>
Previous crash logs	<code>kubectl logs <pod-name> -n teammate-prod --previous</code>
Debug a pod	<code>kubectl describe pod <pod-name> -n teammate-prod</code>
Open shell in pod	<code>kubectl exec -it <pod-name> -n teammate-prod -- bash</code>
Rollback deployment	<code>kubectl rollout undo deployment/reporting-service -n teammate-prod</code>
Scale to 5 replicas	<code>kubectl scale deployment/reporting-service --replicas=5 -n teammate-prod</code>
Update image (Jenkins does this)	<code>kubectl set image deployment/reporting reporting-service=ecr/img:v2 -n teammate-prod</code>
Watch rolling update	<code>kubectl rollout status deployment/reporting-service -n teammate-prod</code>
List configmaps	<code>kubectl get configmap -n teammate-prod</code>
Check port on EC2	<code>netstat -tlnp grep 8080</code>
SSH to EC2	<code>ssh -i mykey.pem ec2-user@<public-ip></code>

End of Document — Teammate Audit Application DevOps Guide